

TALLER DE PROGRAMACION
(75.29) CURSO BUCHWALD - GENENDER

Trabajo Práctico: Los Algoritmos Greedy son juegos de niños

11 de septiembre de 2026

Integrantes

Alumno	Padrón
Michael Mena	102685
Ramiro Raffo	110678
Lucas Ferreira	106406

Índice

1. Análisis del problema	3
2. Algoritmo y Análisis de Complejidad	4
2.1. Implementación	4
2.2. Complejidad temporal y espacial	4
2.3. Variabilidad de los valores	5
3. Demostración de Correctitud	6
3.1. Desigualdad por ronda	6
3.2. Comparación de los puntajes totales	6
4. Impacto de la variabilidad en la optimalidad	7
4.1. Ejemplo con valores muy diferentes	7
4.2. Por qué la variabilidad no rompe la estrategia	7
4.3. Alcance de la conclusión	7
5. Ejemplos de ejecución	9
5.1. Ejemplo 1: Configuración general	9
5.2. Ejemplo 2: Valores crecientes	9
5.3. Ejemplo 3: Valor grande protegido en el interior	9
5.4. Validación automatizada	10
6. Mediciones empíricas y cuadrados mínimos (Punto 6)	11
6.1. Metodología y entorno de pruebas	11
6.2. Ajuste por cuadrados mínimos	11
6.3. Resultados	11
6.4. Análisis de los resultados	13
7. Conclusiones	14
8. Anexos	15

1. Análisis del problema

Se dispone una fila de n monedas y, en cada turno, sólo se puede retirar una moneda de uno de sus extremos. Sophia comienza el juego y también decide qué moneda toma Mateo. Por lo tanto, Sophia busca maximizar su suma y, simultáneamente, minimizar la suma de Mateo.

La estrategia greedy toma una decisión local en cada turno: Sophia elige para sí misma la moneda de mayor valor entre los dos extremos disponibles. En el turno siguiente elige para Mateo la moneda de menor valor entre los extremos. Este criterio se repite hasta retirar todas las monedas.

En cada ronda completa, Sophia toma el máximo de los extremos disponibles y Mateo el mínimo de los nuevos extremos. Por ello, la estrategia favorece a Sophia en cada decisión local. Si los valores se repiten, puede haber empate aunque no todas las monedas sean iguales; por ejemplo, la entrada $[1, 2, 2, 1]$ produce un empate. En general, con valores positivos se garantiza $S \geq M$, y con valores distintos se obtiene una victoria estricta.

2. Algoritmo y Análisis de Complejidad

Se mantienen dos punteros, *izq* y *der*, que representan los extremos aún disponibles. En el turno de Sophia se compara *monedas[izq]* con *monedas[der]* y se retira el mayor. En el turno de Mateo se retira el menor. El puntero del extremo elegido se actualiza y el proceso termina cuando *izq* > *der*.

2.1. Implementación

El algoritmo devuelve la secuencia de elecciones y los puntajes acumulados. La interfaz de consola utiliza esa información para mostrar cada turno.

```
1 def juego_monedas_greedy(monedas):
2     """Devuelve elecciones y puntajes usando el criterio greedy."""
3     if not monedas:
4         raise ValueError("La entrada debe contener al menos una moneda")
5     if any(moneda <= 0 for moneda in monedas):
6         raise ValueError("Los valores de las monedas deben ser positivos")
7
8     elecciones = []
9     izq = 0
10    der = len(monedas) - 1
11    puntaje_sophia = 0
12    puntaje_mateo = 0
13
14    while izq <= der:
15        es_turno_sophia = len(elecciones) % 2 == 0
16        toma_izquierda = (
17            monedas[izq] >= monedas[der]
18            if es_turno_sophia
19            else monedas[izq] <= monedas[der]
20        )
21        if toma_izquierda:
22            posicion = "izquierda"
23            moneda = monedas[izq]
24            izq += 1
25        else:
26            posicion = "derecha"
27            moneda = monedas[der]
28            der -= 1
29
30        jugador = "Sophia" if es_turno_sophia else "Mateo"
31        elecciones.append((f"{{jugador}} ({{posicion}})", moneda))
32        if es_turno_sophia:
33            puntaje_sophia += moneda
34        else:
35            puntaje_mateo += moneda
36
37    return elecciones, puntaje_sophia, puntaje_mateo
```

Listing 1: Implementación del algoritmo greedy

2.2. Complejidad temporal y espacial

La complejidad temporal del algoritmo es $\mathcal{O}(n)$, donde n es la cantidad inicial de monedas.

- La inicialización de los punteros, el turno y los acumuladores toma tiempo constante, $\mathcal{O}(1)$.
- En cada iteración del ciclo `while` se determina el turno y se comparan los dos extremos disponibles.
- Cada iteración retira exactamente una moneda: uno de los punteros avanza o retrocede una posición. Por lo tanto, el ciclo se ejecuta exactamente n veces.

- Las lecturas, comparaciones, sumas y actualizaciones realizadas en una iteración son operaciones de tiempo constante, $\mathcal{O}(1)$.

En consecuencia, el tiempo total es $\mathcal{O}(n)$ y, más precisamente, $\Theta(n)$, ya que el algoritmo siempre procesa todas las monedas.

Si sólo se necesitaran los puntajes, el algoritmo utilizaría espacio auxiliar $\mathcal{O}(1)$: mantendría únicamente los dos punteros, el turno y los acumuladores. La implementación entregada también conserva la secuencia de elecciones para informarla, por lo que su espacio adicional es $\mathcal{O}(n)$. No se utilizan matrices ni llamadas recursivas.

2.3. Variabilidad de los valores

La variabilidad de los valores no modifica la complejidad temporal del algoritmo. Las comparaciones y sumas se consideran operaciones de tiempo constante, independientemente de que los valores sean cercanos, muy dispares, ordenados, desordenados o iguales.

Además, no existen cortes tempranos: el juego debe retirar todas las monedas. Aunque los valores determinan qué rama de cada comparación se ejecuta, ambas ramas realizan la misma cantidad asintótica de trabajo: una suma y el avance de uno de los punteros. Por lo tanto, cualquier entrada de tamaño n produce n iteraciones y el algoritmo mantiene un rendimiento $\Theta(n)$ frente a cualquier distribución de valores.

3. Demostración de Correctitud

Para demostrar la propiedad del algoritmo, dividimos el juego en rondas. Una ronda está formada por el turno de Sophia seguido inmediatamente por el turno de Mateo. Sean C_{izq} y C_{der} las monedas que se encuentran en los extremos al comienzo de una ronda.

3.1. Desigualdad por ronda

Teorema. En toda ronda, la moneda que obtiene Sophia tiene un valor mayor o igual que la moneda que obtiene Mateo.

Demostración. Sin pérdida de generalidad, supongamos que $C_{izq} \geq C_{der}$. En su turno, Sophia toma C_{izq} por ser la mayor de las dos monedas disponibles, de modo que su ganancia en la ronda es $S_k = C_{izq}$. Luego, los extremos disponibles son C_{izq+1} y C_{der} . Sophia elige para Mateo la menor de estas monedas, por lo que

$$M_k = \min(C_{izq+1}, C_{der}) \leq C_{der}.$$

Como $C_{izq} \geq C_{der}$, se obtiene

$$S_k = C_{izq} \geq C_{der} \geq \min(C_{izq+1}, C_{der}) = M_k.$$

El caso $C_{der} \geq C_{izq}$ es análogo, intercambiando los roles de los extremos. Por lo tanto, $S_k \geq M_k$ en toda ronda. $\square$

3.2. Comparación de los puntajes totales

Si n es par, todas las monedas se agrupan en rondas completas y, por la desigualdad anterior,

$$\sum_k S_k \geq \sum_k M_k.$$

Si n es impar, queda una moneda luego de las rondas completas. Esa moneda la toma Sophia, por lo que se agrega un valor no negativo al puntaje de Sophia y se mantiene la misma desigualdad total.

Cuando los valores de las monedas son todos distintos, en cada ronda completa la primera desigualdad entre los extremos es estricta. En consecuencia, $S_k > M_k$ en cada ronda completa y Sophia gana estrictamente. Si los valores pueden repetirse, la conclusión general es $S \geq M$: puede haber empate aun cuando no todas las monedas tengan el mismo valor. Por ejemplo, la entrada $[1, 2, 2, 1]$ produce empate con esta estrategia. El caso de una cantidad par de monedas con el mismo valor es, en particular, necesariamente un empate.

Así, bajo la hipótesis de valores distintos utilizada por el enunciado, el algoritmo garantiza la victoria de Sophia; con valores repetidos, garantiza al menos que Sophia no obtenga un puntaje menor.

4. Impacto de la variabilidad en la optimalidad

En el punto 3 se analizó que la variabilidad numérica de las monedas no afecta el tiempo de ejecución: para una entrada de tamaño n , el algoritmo siempre realiza exactamente n iteraciones. En este punto analizamos si la magnitud o el orden de los valores puede afectar la optimalidad de la estrategia.

4.1. Ejemplo con valores muy diferentes

Consideremos la entrada

$$\text{Monedas} = [50, 10, 2, 1000].$$

El algoritmo se comporta de la siguiente manera:

1. Sophia compara 50 y 1000 y toma 1000.
2. Los extremos restantes son 50 y 2. Sophia elige para Mateo el menor, 2.
3. Sophia compara 50 y 10 y toma 50.
4. Mateo toma la última moneda, 10.

Los puntajes son $S = 1050$ y $M = 12$. El ejemplo muestra que una gran variación entre los valores no perjudica la estrategia. Sin embargo, por sí solo no constituye una demostración de correctitud: esa propiedad se obtiene del argumento general por rondas presentado en la sección anterior.

4.2. Por qué la variabilidad no rompe la estrategia

La demostración de correctitud sólo utiliza comparaciones entre valores. En cada ronda, Sophia toma el máximo de los dos extremos iniciales y luego elige para Mateo el mínimo de los dos extremos que quedan. Si los valores son positivos, se cumple siempre

$$S_k \geq M_k.$$

Esta desigualdad no depende de que los valores sean cercanos, muy dispares, ordenados, desordenados o sigan una distribución particular. Por lo tanto, la variabilidad numérica no altera la propiedad que permite comparar los puntajes totales.

Bajo la hipótesis del enunciado de valores distintos, la desigualdad es estricta en cada ronda completa: los extremos iniciales son distintos y Sophia toma el mayor. En consecuencia, Sophia obtiene un puntaje total mayor que Mateo, salvo el caso de empate excluido por la consigna. Si se permiten valores repetidos, la conclusión general es $S \geq M$ y puede existir empate; por ejemplo, $[1, 2, 2, 1]$ produce empate con esta estrategia.

4.3. Alcance de la conclusión

La optimalidad analizada corresponde al objetivo del enunciado: asegurar que Sophia gane, no maximizar la diferencia entre los puntajes. Como Sophia controla ambos turnos, no existe un oponente que pueda cambiar la elección de Mateo y aprovechar una moneda grande que haya quedado en un extremo.

Por lo tanto, para valores positivos y distintos, ninguna distribución u orden de los valores hace fallar al algoritmo respecto del objetivo de ganar. Si se permitieran valores negativos, habría que revisar la prueba: en una entrada impar, la última moneda ya no necesariamente aumenta el

puntaje de Sophia. Asimismo, maximizar la diferencia $S - M$ sería un objetivo distinto y requeriría un análisis separado; no es necesario para resolver este trabajo.

5. Ejemplos de ejecución

Para corroborar la lógica analizada en los puntos anteriores, presentamos pruebas de escritorio con diferentes configuraciones iniciales de la fila de monedas. En cada caso se detallan las decisiones de Sophia, que controla ambos turnos, y los puntajes parciales.

5.1. Ejemplo 1: Configuración general

Consideremos la fila inicial

$$\text{Fila inicial} = [15, 3, 20, 8, 12].$$

- **Ronda 1:** Sophia compara 15 y 12 y toma 15. Los extremos restantes son 3 y 12; elige para Mateo el menor, 3. Los puntajes quedan $S = 15$ y $M = 3$.
- **Ronda 2:** Sophia compara 20 y 12 y toma 20. Luego compara 8 y 12 para el turno de Mateo y elige 8. Los puntajes quedan $S = 35$ y $M = 11$.
- **Ronda 3:** queda una única moneda, 12, que toma Sophia. El resultado final es $S = 47$ y $M = 11$; Sophia gana.

5.2. Ejemplo 2: Valores crecientes

Analicemos un caso borde con valores estrictamente ordenados:

$$\text{Fila inicial} = [2, 4, 6, 8, 10, 12].$$

- **Ronda 1:** Sophia toma 12 y elige para Mateo el 2. Los puntajes quedan $S = 12$ y $M = 2$.
- **Ronda 2:** Sophia toma 10 y elige para Mateo el 4. Los puntajes quedan $S = 22$ y $M = 6$.
- **Ronda 3:** Sophia toma 8 y queda una única moneda, 6, para Mateo. El resultado final es $S = 30$ y $M = 12$; Sophia gana.

Este caso muestra que el orden creciente de los valores no modifica la decisión greedy ni impide obtener la victoria.

5.3. Ejemplo 3: Valor grande protegido en el interior

Consideremos una moneda de valor muy alto ubicada en una posición interna:

$$\text{Fila inicial} = [100, 2000, 5, 10].$$

- **Ronda 1:** Sophia compara 100 y 10 y toma 100. Los extremos restantes son 2000 y 10; para el turno de Mateo elige 10, que es el menor. Los puntajes quedan $S = 100$ y $M = 10$.
- **Ronda 2:** Sophia compara 2000 y 5 y toma 2000. Queda una única moneda, 5, que toma Mateo. El resultado final es $S = 2100$ y $M = 15$; Sophia gana.

Este ejemplo ilustra que Sophia puede reservar para sí una moneda grande que estaba en el interior, porque controla la elección de Mateo y selecciona el menor extremo disponible en su turno.

5.4. Validación automatizada

Además de las pruebas de escritorio, se implementaron tests automatizados para verificar el contrato de entrada y el comportamiento del algoritmo. Se comprueban los separadores aceptados, el rechazo de valores no positivos, los casos borde con una o dos monedas, cantidades pares e impares, valores repetidos y la generación de una elección por cada moneda.

También se verifica que cada elección corresponda a uno de los extremos disponibles y que Sophia tome el máximo mientras que, en el turno siguiente, Mateo reciba el mínimo. Finalmente, se comprueba que la suma de los puntajes sea igual a la suma de las monedas y que, para valores positivos, se cumpla $S \geq M$. Los casos provistos por la cátedra se ejecutan en una suite separada de validación funcional.

6. Mediciones empíricas y cuadrados mínimos (Punto 6)

El objetivo de este experimento es contrastar empíricamente la complejidad temporal $\Theta(n)$ obtenida en el punto 3 y observar si la distribución de los valores modifica el tiempo de ejecución.

6.1. Metodología y entorno de pruebas

Las mediciones se realizaron con Python y se tomó el tiempo únicamente durante la ejecución del algoritmo, usando `time.perf_counter()`. Cada punto se midió tres veces y se utilizó la mediana para reducir el efecto de procesos externos y otros ruidos del sistema. Se compararon dos variantes:

1. **Sólo puntajes:** ejecuta el núcleo greedy y conserva únicamente los dos acumuladores. Su espacio auxiliar es $\mathcal{O}(1)$.
2. **Completa:** además de calcular los puntajes, conserva la secuencia de elecciones requerida por la interfaz. Su espacio auxiliar es $\mathcal{O}(n)$.

Para cada variante se midieron tres tipos de entrada: valores aleatorios entre 1 y 1000, valores estrictamente crecientes y valores todos iguales a 10. Se usaron veinte tamaños desde 50,000 hasta 1,000,000, con incrementos de 50,000. Este rango fue elegido para que las mediciones pudieran ejecutarse en el entorno disponible sin superar varios minutos ni consumir una cantidad excesiva de memoria. El algoritmo sigue siendo lineal para tamaños mayores, pero no se incluyeron por razones prácticas de tiempo y memoria.

6.2. Ajuste por cuadrados mínimos

Siguiendo el enfoque presentado por la cátedra, se ajustaron los tiempos a una función lineal con ordenada al origen:

$$f(n) = an + b.$$

El término b permite representar el costo fijo de iniciar la medición. Para los puntos (n_i, t_i) , los parámetros se obtienen minimizando

$$\sum_i (t_i - (an_i + b))^2.$$

Además del gráfico de tiempos, se calcularon los residuos absolutos y la suma de errores cuadráticos (SSE) para evaluar el ajuste.

Los archivos provistos por la cátedra en `tests_catedra/` se utilizaron como validación funcional independiente. Sus tamaños son casos particulares y no forman parte del ajuste temporal. Además, el archivo de resultados advierte que los puntajes pueden variar según la estrategia y que lo importante es que Sophia gane; por ese motivo se verificó la victoria y la cantidad de elecciones, sin exigir igualdad con un puntaje de referencia.

6.3. Resultados

La Figura 1 muestra los tiempos medidos. Las líneas continuas corresponden al núcleo que conserva sólo puntajes y las líneas discontinuas a la variante completa. El color identifica el tipo de entrada.

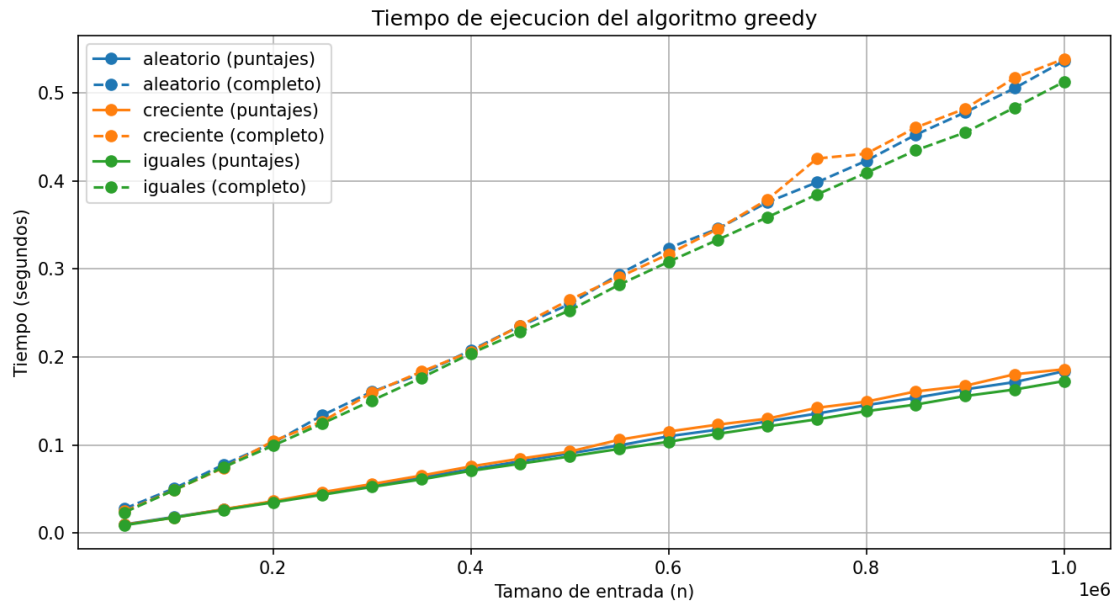


Figura 1: Tiempos de ejecución para ambas variantes y distribuciones.

La Figura 2 presenta el error absoluto de cada punto respecto de su ajuste lineal.

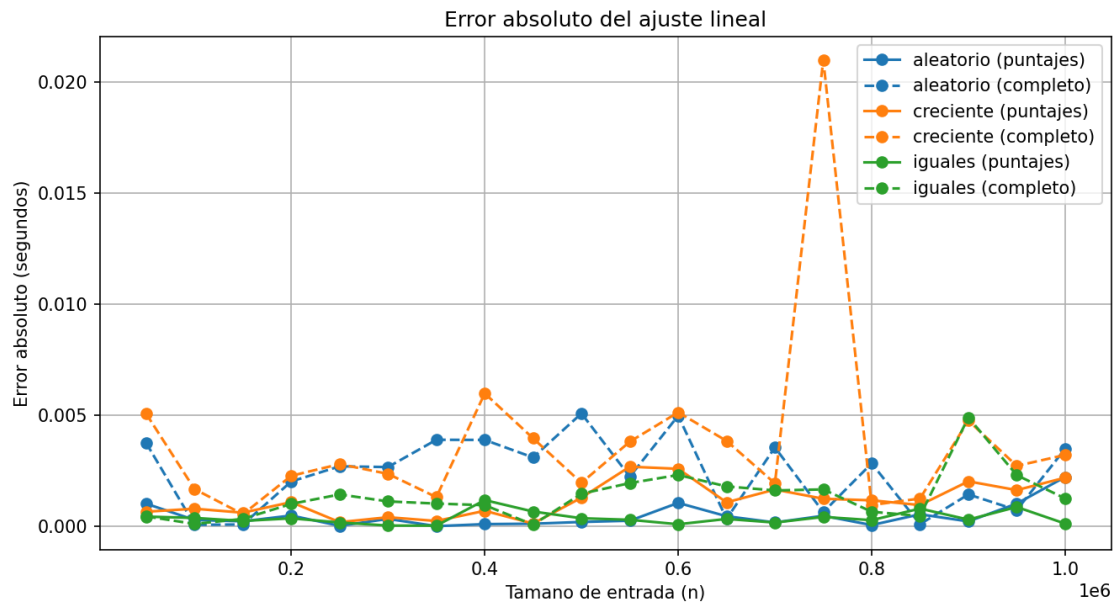


Figura 2: Residuos absolutos de los ajustes lineales.

En la ejecución registrada se obtuvieron los siguientes parámetros, donde a está expresado en segundos por moneda:

Caso y variante	a	b	SSE
Aleatorio, puntajes	$1,818 \times 10^{-7}$	$-2,654 \times 10^{-4}$	$9,536 \times 10^{-6}$
Aleatorio, completa	$5,355 \times 10^{-7}$	$-2,767 \times 10^{-3}$	$1,639 \times 10^{-4}$
Creciente, puntajes	$1,887 \times 10^{-7}$	$-5,778 \times 10^{-4}$	$3,822 \times 10^{-5}$
Creciente, completa	$5,495 \times 10^{-7}$	$-7,808 \times 10^{-3}$	$6,467 \times 10^{-4}$
Iguales, puntajes	$1,717 \times 10^{-7}$	$7,733 \times 10^{-4}$	$4,517 \times 10^{-6}$
Iguales, completa	$5,137 \times 10^{-7}$	$-2,560 \times 10^{-3}$	$5,803 \times 10^{-5}$

6.4. Análisis de los resultados

Las pendientes son aproximadamente constantes dentro de cada variante y los tiempos crecen linealmente con n , en acuerdo con la complejidad teórica $\Theta(n)$. La variante completa es más lenta porque construye y almacena una tupla por cada elección, además de ejecutar las comparaciones y acumulaciones del núcleo.

Las diferencias entre casos aleatorios, crecientes e iguales afectan las constantes medidas y el ruido del experimento, pero no la cantidad de iteraciones: siempre se procesa una vez cada moneda. Por ello, la distribución de valores no cambia la complejidad asintótica. Los resultados respaldan la conclusión teórica, dentro del rango de tamaños utilizado.

7. Conclusiones

El algoritmo greedy recorre la fila una sola vez y mantiene únicamente sus dos extremos. Su costo lineal no depende de la magnitud ni de la variabilidad de los valores: esas características sólo pueden cambiar cuál de los dos extremos se elige, pero no la cantidad de iteraciones. La implementación permite verificar tanto la secuencia de decisiones como los puntajes finales de ambos jugadores.

8. Anexos

Esta sección queda reservada para incorporar correcciones y material adicional en caso de una reentrega.